

第七单元：字符串

1. 创建字符串

2. 字符串获取

2.1. 字符串 length

2.2. 通过索引访问字符串中的字符

3. 字符串常用方法

1. indexOf()

2. lastIndexOf()

3. includes()

4. slice()

5. substring()

6. substr()

7. split()

8. concat()

9. replace()

10. toLowerCase()

11. toUpperCase()

12. trim()

13. at()

14. charAt()

15. charCodeAt()

16. fromCharCode()

4. 总结

1. 创建字符串

- 示例

```
1 var str1 = 'abc' // 字面量方式
2 var str2 = new String('abc') // 构造函数方式
```

2. 字符串获取

2.1. 字符串 length

- length 返回字符串的长度
- 字符串的 length 不可以设置，只能获取

```
1 var str = 'abcdefg'
2 console.log(str.length) // 7
```

2.2. 通过索引访问字符串中的字符

```
1 var str = 'abcdefg'
2 console.log(str[0]) // a
3 console.log(str[1]) // b
4 console.log(str.charAt(2)) // c
```

3. 字符串常用方法

注意：没有任何方法可以修改原字符串！！！！

1. indexOf()

- 语法：string.indexOf(searchValue[, fromIndex])
- 功能：返回子串在字符串中首次出现的位置，未找到返回-1。
- 参数：

- **searchValue**: 要查找的子串。
- **fromIndex** (可选): 开始查找的位置。
- **示例:**

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.indexOf("World")); // 输出 6
```

2. lastIndexOf()

- **语法:** `string.lastIndexOf(searchValue[, fromIndex])`
- **功能:** 返回子串在字符串中最后一次出现的位置, 未找到返回-1。
- **参数:**
 - **searchValue**: 要查找的子串。
 - **fromIndex** (可选): 开始查找的位置。
- **示例:**

JavaScript |

```
1 let str = "Hello World, Hello Earth";
2 console.log(str.lastIndexOf("Hello")); // 输出 13
```

3. includes()

- **语法:** `string.includes(searchString[, position])`
- **功能:** 判断字符串是否包含指定的子串, 返回布尔值。
- **参数:**
 - **searchString**: 要查找的子串。
 - **position** (可选): 开始查找的位置。
- **示例:**

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.includes("World")); // 输出 true
```

4. slice()

- 语法: `string.slice(beginIndex[, endIndex])`
- 功能: 提取字符串的一部分, 返回新字符串, 不改变原字符串。
- 参数:
 - `beginIndex` – 开始提取的位置 (包括)。
 - `endIndex` (可选) – 结束提取的位置 (不包括)。
- 示例:

JavaScript

```
1 let str = "Hello World";
2 console.log(str.slice(1, 5)); // 输出 'ello'
```

- 注意事项: 如果`beginIndex`为负值, 则从字符串尾部开始计算位置。

5. substring()

- 语法: `string.substring(indexStart[, indexEnd])`
- 功能: 返回位于字符串中指定位置之间的子字符串。
- 参数:
 - `indexStart` – 要提取的第一个字符的索引。
 - `indexEnd` (可选) – 在其中停止提取的字符的索引 (不包括)。
- 示例:

JavaScript

```
1 let str = "Hello World";
2 console.log(str.substring(1, 5)); // 输出 'ello'
```

- 注意事项: 与`slice`相比, `substring`不接受负的索引值。

6. substr()

- 语法: `string.substr(start[, length])`
- 功能: 根据指定的开始位置和长度返回一个新的子字符串。
- 参数:

- **start** – 子字符串开始的位置。负值从字符串末尾开始计数。
- **length** (可选) – 子字符串的长度。

- 示例:

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.substr(1, 4)); // 输出 'ello'
```

7. split()

- 语法: `string.split(separator[, limit])`
- 功能: 根据分隔符将字符串分割为数组。
- 参数:
 - **separator** – 指定用于分割字符串的字符或正则表达式。
 - **limit** (可选) – 返回的数组的最大长度。

- 示例:

JavaScript |

```
1 let str = "Hello World";
2 let words = str.split(" ");
3 console.log(words); // 输出 ['Hello', 'World']
```

- 注意事项: 如果没有提供分隔符, 整个字符串将作为单一元素返回在数组中。

8. concat()

- 语法: `string1.concat(string2, ..., stringN)`
- 功能: 合并两个或多个字符串。
- 示例:

JavaScript |

```
1 let str1 = "Hello";
2 let str2 = " World";
3 console.log(str1.concat(str2)); // 输出 'Hello World'
```

9. replace()

- 语法: `string.replace(regex|substr, newSubstr|function)`
- 功能: 替换与正则表达式或子串匹配的部分。
- 参数:
 - `regex|substr` – 要被替换的子串或正则表达式。
 - `newSubstr|function` – 替换的字符串或进行替换的函数。
- 示例:

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.replace("World", "Earth")); // 输出 'Hello Earth'
```

- 注意事项: 默认只替换第一次出现的匹配项, 要替换所有匹配项需使用正则表达式配合全局标志 (g) 。

10. toLowerCase()

- 语法: `string.toLowerCase()`
- 功能: 将字符串转换为小写。
- 示例:

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.toLowerCase()); // 输出 'hello world'
```

11. toUpperCase()

- 语法: `string.toUpperCase()`
- 功能: 将字符串转换为大写。
- 示例:

JavaScript |

```
1 let str = "Hello World";
2 console.log(str.toUpperCase()); // 输出 'HELLO WORLD'
```

12. trim()

- 语法: `string.trim()`
- 功能: 去除字符串两端的空白字符。
- 示例:

JavaScript

```
1 let str = "  Hello World  ";
2 console.log(str.trim()); // 输出 'Hello World'
```

13. at()

- 语法: `string.at(index)`
- 功能: 返回字符串中指定索引位置的字符。
- 特点: 支持负数索引, 表示从字符串末尾开始计算
- 示例:

JSON

```
1 var str = "hello china";
2 console.log( str.at(1) ) // 'e'
3 console.log( str.at(-1) ) // 'a'
```

14. charAt()

- 语法: `string.charAt(index)`
- 功能: 返回字符串中指定索引位置的字符。
- 特点: 不支持负数, 负数返回空字符串“”。
- 示例:

JSON

```
1 var str = "hello china";
2 console.log( str.charAt(1) ) // e
3 console.log( str.charAt(-1) ) // ''
```

15. charCodeAt()

- 语法: `string.charCodeAt(index)`
- 功能: 返回字符串中指定索引位置字符串的Unicode编码(十进制)。
- 特点: 不支持负数, 负数会返回NaN; 实际应用于处理字符编码相关逻辑(如加密), 兼容性更佳。
- 示例:

▼

JSON |

```

1  var str = "hello china";
2  console.log( str.charCodeAt(1) )  // 101  -> 'e'的Unicode编码
3  console.log( str.charCodeAt(-1) ) // NaN

```

16. fromCharCode()

- 语法: `String.fromCharCode(Unicode)`
- 功能: 返回该Unicode编码对应的字符。
- 示例:

▼

JSON |

```

1  console.log( String.fromCharCode(101) ) // 'e'
2  console.log( String.fromCharCode(65) )  // 'A'
3  console.log( String.fromCharCode(97) )  // 'a'
4  console.log( String.fromCharCode(48) )  // '0'

```

4. 总结

1. 字符串的左右方法都不会修改原字符串
2. 字符串和数组的公共方法: `indexOf()` `lastIndexOf()` `slice()` `concat()` `includes()` `at()`;
3. 字符串转数组`split()`
4. 数组转字符串`join()`
- 5.